

5051 Homework2

LAN Tianwei

tlanaa@connect.ust.hk

October 16, 2025

Question 1

Use `lscpu` command in terminal, then I get the following output:

```
Architecture:                x86_64
  CPU op-mode(s):            32-bit, 64-bit
  Address sizes:             48 bits physical, 48 bits virtual
  Byte Order:                Little Endian
CPU(s):                      12
  On-line CPU(s) list:      0-11
Vendor ID:                   AuthenticAMD
  Model name:                AMD Ryzen 5 5560U with Radeon Graphics
  CPU family:                25
  Model:                     80
  Thread(s) per core:       2
  Core(s) per socket:       6
  Socket(s):                 1
  Stepping:                  0
  BogomIPS:                  4591.26
  Flags:                      ...(too long)
Virtualization features:
  Virtualization:            AMD-V
  Hypervisor vendor:         Microsoft
  Virtualization type:       full
Caches (sum of all):
  L1d:                       192 KiB (6 instances)
  L1i:                       192 KiB (6 instances)
  L2:                         3 MiB (6 instances)
  L3:                         8 MiB (1 instance)
NUMA:
  NUMA node(s):              1
  NUMA node0 CPU(s):         0-11
Vulnerabilities:
  ...
```

(a) Does your CPU use an x86 or ARM architecture (or something else)?

```
Architecture:                x86_64
```

So, x86 arch of course.

(b) How many cores does your CPU have?

```
Vendor ID:                AuthenticAMD
  Model name:              AMD Ryzen 5 5560U with Radeon Graphics
    ...
  Thread(s) per core:     2
  Core(s) per socket:     6
  Socket(s):              1
    ...
```

Number of core(s) = Core(s) per socket * Socket(s), thus there are 6 cores in my CPU.

(c) What are the sizes of your L1, L2 and L3 caches?

```
Caches (sum of all):
  L1d:                    192 KiB (6 instances)
  L1i:                    192 KiB (6 instances)
  L2:                     3 MiB (6 instances)
  L3:                     8 MiB (1 instance)
```

According to online information, I searched and found that modern CPU architecture designs usually divide the L1 cache into instruction (i) and data (d). This design of caching data and instructions separately is called a variant of Harvard architecture. L1 data cache 192 KiB (32 KiB each core), L1 instruction cache 192 KiB (32 KiB each core), L2 3 MiB (0.5 MiB each core), L3 8 MiB (shared by all 6 cores).

(d) What is the size of a cache line on your CPU caches?

The size of a cache line was not shown in the output. Use this command:

```
getconf LEVEL1_DCACHE_LINESIZE
```

Then I get 64 (bytes), which is the standard cache line length for x86-64.

(e) Does your CPU support simultaneous multi threading? If so, how many threads per core?

Yes, my CPU supports SMT with 2 threads per core, according to (c).

(f) Does your CPU support SIMD?

```
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat
pse36 clflush mmx fxsr sse sse2 ht syscall nx mmxext fxsr_opt pdpe1gb
rdtscp lm constant_tsc rep_good nopl tsc_reliable nonstop_tsc cpuid
extd_apicid tsc_known_freq pni pclmulqdq ssse3 fma cx16 sse4_1 sse4_2
movbe popcnt aes xsave avx f16c rdrand hypervisor lahf_lm cmp_legacy
svm cr8_legacy abm sse4a misalignsse 3dnowprefetch osvw topoext
perfctr_core ssbd ibrs ibpb stibp vmmcall fsgsbase bmi1 avx2 smep bmi2
erms invpcid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec
xgetbv1 xsaves clzero xsaveerptr arat npt nrip_save tsc_scale
vmcb_clean flushbyasid decodeassists pausefilter pfthreshold
v_vmsave_vmload umip vaes vpclmulqdq rdpid fsrm
```

Yes, my CPU supports SIMD from SSE series up to AVX2/FMA; 256-bit vectors are available.

Question 2

(a) Would we benefit from having a superscalar core to execute the code above? Explain your reasoning.

This program only contains 2 μ -ops:

```
load  a[i]      ;
add   sum, tmp  ;
```

Each addition must wait for the previous addition to complete, so superscalar cannot improve throughput and will bring almost no performance benefits.

(b) Suppose the bandwidth is such that the CPU performs additions faster than the rate at which data arrives. Estimate how long it will take for the program above to terminate (consider only the for loop part).

Since the CPU can add faster than the memory can deliver data, the loop is bandwidth-bound. Each float element occupies 4 bytes, so the total volume that must be transferred is $4n$ bytes. At a bandwidth of B GB/s ($1GB \approx 10^9 \text{ bytes}$), the estimated transfer time is:

$$T \approx 4n / (B \times 10^9) \quad \text{seconds} = 4n / B \quad \text{nano - seconds}.$$

(c) Assume for some reasons that the data is actually arriving at a rate faster than the CPU can perform additions. How would your answer in (b) change?

Then the loop becomes compute-bound. The time is simply n consecutive add latencies.

$$T \approx n / (F \times 10^9) \quad \text{seconds} = n / F \quad \text{nano - seconds}.$$

(d) How would your answer in (c) change if the CPU only has a finite number of registers? You don't need to do any calculation for this question, just provide a brief explanation.

Finite registers mean that the accumulator must frequently read and write memory, shifting the CPU from compute-bound to bandwidth-bound, potentially slowing it down.

Question 3

(a) Do you think including effects of caches would change the arithmetic intensity of dot product? Explain your reasoning.

The arithmetic intensity of the dot product calculated in lecture was 0.25 FLOP/byte. With cache, once a cache line is loaded, multiple elements within the line are reused, eliminating the need for memory access. This significantly increases arithmetic intensity, potentially shifting from bandwidth-bound to compute-bound.

(b)How about $n \times n$ Matrix multiplication?

Without a cache, each element is accessed only once, and all three matrices flow serially through memory. This results in a computational effort of $2n^3$ *FLOPs* and a main memory traffic of $12n^2$ *bytes* (single-precision). The arithmetic intensity is

$$\approx 2n^3/12n^2 = n/6 \text{ FLOPs/byte}$$

, increasing with n , but still relatively low.

With cache accommodating matrices A and B permanently, the main memory traffic drops from $12n^2$ to $4n^2$ (reading A and B only once, and writing back C once). The intensity immediately increases to

$$2n^3/4n^2 = n/2 \text{ FLOPs/byte}$$

tripling the original value.

(c)Can you devise an alternative matrix multiplication algorithm that can utilize caches better than our naive algorithm above?

I understand the question as removing the “cache-is-large-enough” assumption; after all, if the whole matrices already fit, there is no room to “use the cache better.” With only a cache of size b , we can tile A and B into $b \times b$ blocks. Each such block is fetched once and reused b times before it is evicted, while a $b \times b$ tile of C is accumulated in-cache and written back only when the tile is complete. Under this ideal model the arithmetic intensity becomes $n/2b$.

```
# Assume b is the block size, so 3*b*b can fit into the cache
for ii in range(0, n, b):
    for jj in range(0, n, b):
        for kk in range(0, n, b):
            # Compute sub-block C[ii:ii+b][jj:jj+b]
            for i in range(ii, ii+b):
                for j in range(jj, jj+b):
                    for k in range(kk, kk+b):
                        C[i][j] += A[i][k] * B[k][j]
```

After asking AI, I learned that this is the idea of **tiling**.

Question 4

See *histogram.py*. When I searched for information, I found that `threading.Lock()` can be used to obtain thread locks to avoid data inconsistencies caused by multiple threads modifying shared resources at the same time. Specifically, in this question, it is when each thread writes a histogram. The reference webpage is <https://gairuo.com/p/python-threading-lock-object>

```
Results match: True
Serial time: 0.0061s
Parallel time: 0.0143s
Speed up: 2.36x
```